



Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas -
ICEB Departamento de Computação -
DECOM

Disciplina: BCC 138 Programação Orientada a
Objetos

Trabalho Prático

Instruções

- O problema deve ser resolvido por meio de um programa em C++ e a STL.
- O código-fonte deve estar devidamente comentado.
- Não serão aceitos trabalhos que caracterizem cópia (mesma estrutura e algumas pequenas modificações) de outro ou de códigos da internet
- Eventualmene, após a entrega dos trabalhos serão marcadas entrevistas com cada um dos aluno para apresentação dos mesmos para o professor

Entrega

- A entrega do código-fonte será feito pelo Moodle
- Deve ser entregue um zip com:
 - Relatório (descrito ao fim deste texto)
 - Código fonte
 - Descrição de como compilar e executar o projeto pela linha de comando
 - Vídeo mostrando a compilação, execução e uso do código.

Avaliação

- Funcionamento adequado do programa
 - Códigos que não compilarem e tiveram warning mininuirão a nota.
 - Corretudo (independente se gerado por IDE ou manualmente)
- Atendimento ao enunciado do trabalho
- Comentários
- Identação do código e boas práticas de programação
- Boa organização do código fonte em geral
- Bom uso da STL

1. Trabalho

Implementar uma estrutura de classes que permita armazenar e manipular dados de ;exércitos, unidades e batalhas. Essa estrutura será utilizada para montar um protótipo de jogo de simulação de estratégia, no qual o jogador assume o papel de um comandante, organizando seu exército e um software simula as batalhas entre os exércitos.

Existem vários jogos de estratégia de guerra como Warhammer 40k, Age of Empires, e Starcraft, todos com o mesmo estilo de organização de unidades e combate estratégico.

Nessa parte, você deve se preocupar APENAS com a implementação orientada a objetos que envolve cada entidade do problema. Um exemplo mínimo é fornecido na Figura 1, que inclui as classes (que devem ser obrigatoriamente em quantidade e nome igual ao do diagrama), bem como seus atributos e métodos. Você está livre para criar mais métodos, atributos e construtores. Mas cuidado para não exagerar.

Repare que o diagrama indica a relação entre as classes, entre as quais temos herança (é-um) e uso (dependência).

2. Requisitos

Classe Unidade e derivadas

- **Poder de Ataque:** o método ***getPoderAtaque()*** irá retornar um número inteiro de 0 a 100 com o poder de ataque daquela unidade. Um objeto Unidade tem seu poder de ataque dado pelo atributo protegido ***poderAtaque***. No entanto, cada subclasse deverá sobrescrever o método ***getPoderAtaque()*** de forma a calcular o poder de ataque de acordo com seus próprios atributos. Para os valores dos atributos deve-se gerar números aleatórios entre os intervalos abaixo:
 - Infantaria:
 - poderAtaque: 30 a 60
 - força: 20 a 50
 - velocidade: 10 a 40
 - Veiculo:
 - poderAtaque: 40 a 80
 - blindagem: 30 a 70

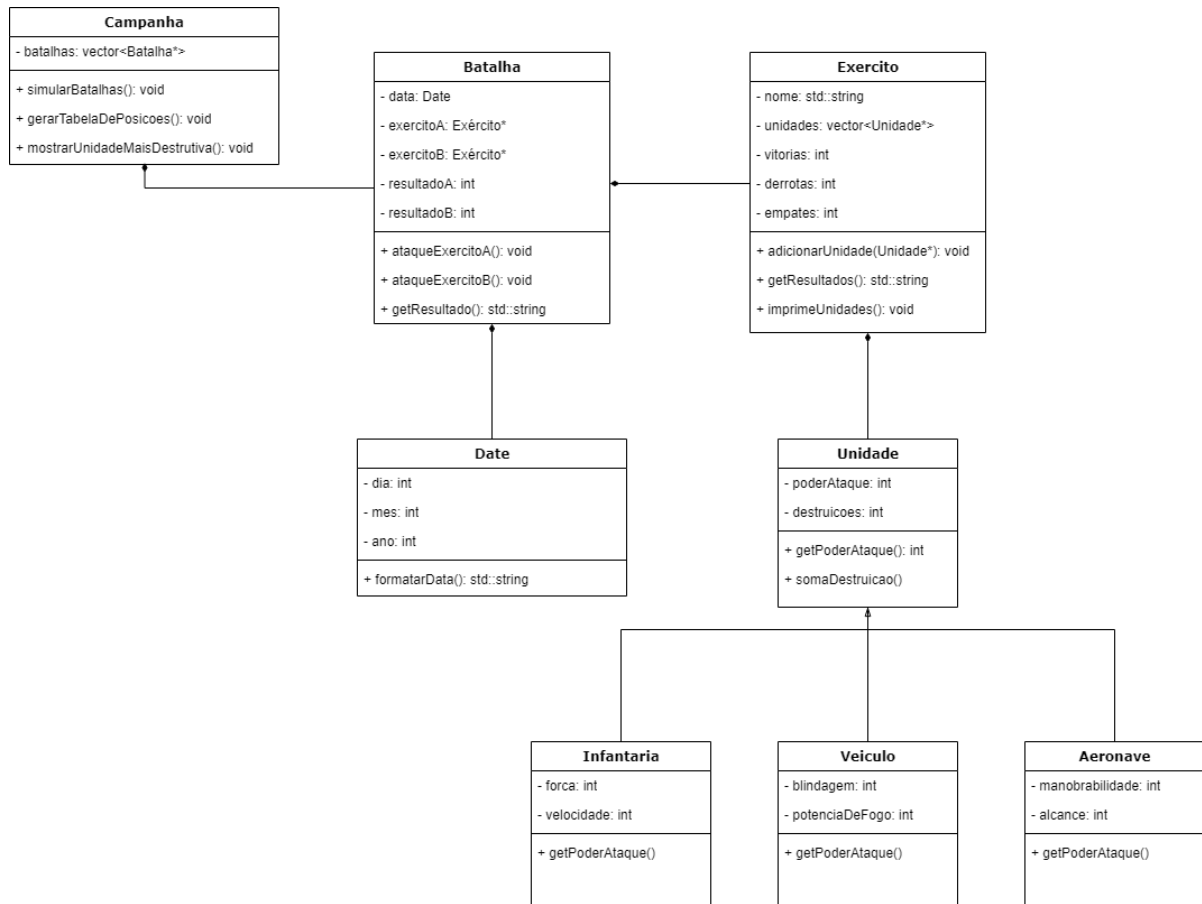
- potênciadeFogo: 20 a 50
- Aeronave:
 - poderAtaque: 50 a 90
 - manobrabilidade: 30 a 70
 - alcance: 20 a 50
- O cálculo do poder de ataque deverá ser feito da seguinte forma:
 - Infantaria: $((\text{poderAtaque} * 5) + (\text{força} * 3) + (\text{velocidade} * 2)) / 10$
 - Veículo: $((\text{poderAtaque} * 5) + (\text{blindagem} * 4) + (\text{potênciaDeFogo} * 1)) / 10$
 - Aeronave: $((\text{poderAtaque} * 5) + (\text{manobrabilidade} * 3) + (\text{alcance} * 2)) / 10$
- *Destruições*: o método **somaDestruicao()** deverá somar uma destruição ao registro de destruições da unidade, que é inicializado com 0 e vai aumentando conforme a unidade destrói oponente em uma batalha.

Classe Exército:

- **Construtor**: o construtor de Exército recebe apenas o nome do exército. As unidades são inseridas posteriormente. Nessa primeira implementação não há remoção de unidades.
- **Histórico de Batalhas**: o número de vitórias, derrotas e empates é inicializado com 0 e aumenta conforme o resultado das batalhas.
- **Métodos**:
 - **getResultados()**: retorna uma string com os resultados de um exército, no formato: Vitórias: 0, Empates: 0, Derrotas: 0.
 - **imprimeUnidades()**: imprime todas as unidades do exército.

Classe Batalha:

- **Construtor**: recebe uma data e dois exércitos.
- **Métodos**:
 - **ataqueExercitoA()**: simula um ataque do exército A.
 - **ataqueExercitoB()**: simula um ataque do exército B.
 - **getResultado()**: retorna uma string com o resultado da batalha, no formato: Nome do Exército A 0 x 0 Nome do Exército B.



3. Instruções

Todas as classes devem ser implementadas com separação da interface .h e da implementação .cpp. Deverá ser criado um Makefile para compilar o projeto.

- Como em C++ não há um tipo Date nativo, você poderá criar um tipo ou uma classe Date ou fazer uma adaptação usando os tipos disponíveis.
- Implemente os destrutores conforme a necessidade de cada classe.
- Crie uma classe Campanha que simule batalhas de ida e volta. Para cada batalha, instancie um objeto da classe Batalha que será composto de dois exércitos. Cada objeto do Exército contém cinco objetos Unidade (entre os quais: duas unidades de infantaria, duas de veículos e uma aeronave).
- Nesta primeira etapa, para determinar qual exército ganha a batalha, gere aleatoriamente os poderes de ataque de cada uma das unidades. O exército com a maior soma de poderes de ataque será o ganhador. A cada nova batalha, os poderes das unidades devem

ser sorteados novamente.

- Crie um método que gere a tabela de posições, a tabela deve poder ser mostrada na tela ou salva em um arquivo.
- Crie um método que mostre quem é a unidade com mais destruições na campanha.

O trabalho consiste em implementar um programa em C++ seguindo as instruções listadas na parte anterior, as classes do programa devem ter alta coesão e baixo acoplamento e deve-se explorar os conceitos de abstração, herança e polimorfismo, para tal deverá ser feito uso de herança e classes abstratas.

Deverão ser entregues os seguintes itens compactados em um zip:

- Relatório
- Código fonte

O relatório deve ter:

- A descrição da arquitetura e implementação do trabalho em C++ através de diagramas da UML explicando detalhadamente como cada item foi construído,
- Quais foram as decisões de projeto, quais os recursos de linguagem foram utilizados, etc.
- O relatório deverá ter também as instruções de compilação e execução do programa utilizando terminal ou prompt de comando, com exemplos.

O Código-fonte do programa deve ter:

- Comentários descrevendo os detalhes de implementação.
- Deve-se seguir as boas práticas de escrita de código como: Nomenclatura, como CamelCase.
- Alinhamento.
- Boa organização do código em geral para facilitar a leitura